TREE DATA STRUCTURES

SHORT NOTES | EXAM REVISION EDITION

A concise, corrected and clearly organized revision guide

HOW TO USE THESE NOTES

Grey boxes contain examples and diagrams. Blue boxes highlight formulas. Yellow boxes contain exam reminders. Green boxes highlight definitions and key results. The content stays concise for quick revision.

Prepared from the supplied short notes

1. Basic Tree Terminologies

A tree is a non-linear data structure made up of nodes connected by edges.

DEFINITION — NULL TREE

A tree containing zero nodes.

No.	Terminology	Short Definition
1	NULL Tree	A tree containing zero nodes.
2	Root Node	The topmost node of a tree; it has no parent.
3	Node	An individual element that stores data in a tree.
4	Parent Node	A node that has one or more children.
5	Child Node	A node directly connected below its parent.
6	Leaf Node	A node having zero children; also called a terminal node.
7	Non-Leaf Node	A node having at least one child; also called an internal node.
8	Degree of a Node	Total number of direct children of a node.
9	Degree of a Tree	Maximum degree among all nodes in a tree.
10	Level of a Node	Number of edges from the root to that node.
11	Height of a Tree	Maximum number of edges on the longest root-to-leaf path.
12	Path	A sequence of consecutively connected nodes.
13	Ancestor Node	Any node above a particular node on its path to the root.
14	Descendant Node	Any node reachable below a particular node.
15	Siblings	Nodes having the same parent.
16	Directed Edge	A connection directed from parent to child.
17	In-Degree	Number of incoming edges to a node.
18	Out-Degree	Number of outgoing edges from a node.

COUNT CORRECTION

The original heading said 16 terminologies, but the table contains 18. The corrected count is 18.

1.1 Tree Properties and Important Values

Terminology	Formula / Meaning
Total Nodes	8 (for the example tree in the original notes)
Total Edges	Nodes − 1 = 7 (for a tree with 8 nodes)
Degree of Node	Number of direct children
Degree of Tree	Maximum degree of any node
Level of Root	0
Level of Child	Parent level + 1
Height of Tree	Maximum level, when root is at Level 0
Leaf Node	Degree = 0
In-Degree of Root	0
In-Degree of Other Nodes	1
Out-Degree	Number of outgoing edges

IMPORTANT TREE FACTS

Each node can have 0 to n children in a general tree. There is no fixed maximum number of children. In a tree with n nodes, the number of edges is n − 1.

2. General Tree and Binary Tree

2.1 General Tree

- Each node can have 0 to n children.
- There is no fixed maximum number of children.

2.2 Binary Tree

DEFINITION

A binary tree is a tree in which each node can have a maximum of two children: a Left Child and a Right Child.

Type	Explanation
Standard Binary Tree	A normal binary tree with no special ordering rule.
Threaded Binary Tree (TBT)	Uses threads in place of certain NULL pointers.
Binary Search Tree (BST)	Smaller values are placed on the left; greater values (and, in these notes, equal values) are placed on the right.
AVL Tree	A self-balancing Binary Search Tree.

2.3 Binary Tree Operations

1. Creation
2. Traversal
3. Insertion
4. Deletion

5. Merge
6. Copy
7. Compare

3. Binary Tree Memory Representations

Two methods are used to represent a binary tree in memory:

Representation	Explanation
Linked Node	Uses pointers (left, data, right).
Sequential Array	Stores nodes in a one-dimensional array.

ARRAY FORMULAS — 0-BASED INDEXING

For a node stored at array index i:
Left Child = 2i + 1
Right Child = 2i + 2
Parent = floor((i − 1) / 2)

These formulas assume the root is stored at index 0. For the root, the parent formula is not used.

4. Threaded Binary Tree (TBT)

DEFINITION

A binary tree in which NULL pointers are replaced with threads pointing to inorder predecessors or successors.

Purpose: Traversal without recursion or a stack.

Important Fact	Value
Total pointers	2n
NULL pointers in a binary tree with n nodes	n + 1

Type	Thread Direction
Left TBT	Inorder predecessor
Right TBT	Inorder successor
Double TBT	Both predecessor and successor threads

EXAMPLE TREE



THREADING NOTE

A thread uses an otherwise NULL child pointer to link to the inorder predecessor or successor. The exact pointer tagging convention is implementation-dependent.

5. Binary Tree Traversals

Traversal	Rule	Output for the example tree
Inorder	L – V – R	D, B, E, A, C
Preorder	V – L – R	A, B, D, E, C
Postorder	L – R – V	D, E, B, C, A

TRAVERSAL RULES

Inorder: Left → Print → Right
Preorder: Print → Left → Right
Postorder: Left → Right → Print

```
if (root != NULL)
{
    // Recursive calls and printf()
}
```

The original notes include this short recursive-code fragment as a reminder; the exact recursive calls depend on the traversal being performed.

6. Binary Search Tree (BST)

DEFINITION

A Binary Search Tree (BST) is a binary tree in which the left subtree contains smaller values and the right subtree contains greater than or equal values. Both subtrees must also follow the BST property.

BST EXAMPLE



6.1 BST Properties

- Maximum two children per node.
- Left values are smaller than the parent.
- Right values are greater than or equal to the parent.
- Every subtree must also follow the BST property.

DUPLICATE-VALUE RULE

In this set of notes, values equal to a node's value are directed to the right. Keep this rule consistent during BST insertion.

6.2 Inorder Traversal

Rule: Left → Visit → Right (L–V–R)

OUTPUT

20 → 30 → 40 → 50 → 60 → 70 → 80

Inorder traversal produces sorted (non-decreasing) output.

6.3 Searching

1. Start from the root.
2. If equal → Element found.
3. If smaller → Move left.
4. If greater → Move right.
5. If NULL → Element not found.

6.4 Insertion

1. Start from the root.
2. Compare the new value.
3. Smaller → Move left.
4. Greater than or equal → Move right (duplicate-value convention used in these notes).
5. Insert at the NULL position.

6.5 Deletion — Three Cases

Case 1: Leaf Node (No Children)

Simply remove the node.

BEFORE

```
  30
 /  \\
20  40
```

AFTER DELETING 20

```
  30
    \\
     40
```

Case 2: One Child

Connect the parent directly to the node's child.

BEFORE

```
   50
  /
30
  \\
   40
```

AFTER DELETING 30

```
  50
 /
40
```

Case 3: Two Children

Use either the inorder successor or inorder predecessor.

- **Successor:** Smallest value in the right subtree.
- **Predecessor:** Largest value in the left subtree.

Example: Delete 50.

USING INORDER SUCCESSOR 60

```
          60
        /  \\
      30    70
     / \\     \\
    20  40    80
```

USING INORDER PREDECESSOR 40

```
          40
        /  \\
      30    70
     /     /  \\
    20    60  80
```

6.6 Advantages

- Fast searching when balanced.
- Easy insertion and deletion.
- Inorder gives sorted data.
- Supports dynamic memory allocation.

6.7 Disadvantages

- Searching becomes slow if the tree is skewed.
- Requires extra pointer memory.
- Two-child deletion is more complicated.

6.8 Time Complexity

Operation	Average	Worst
Searching	O(log n)	O(n)
Insertion	O(log n)	O(n)
Deletion	O(log n)	O(n)

COMPLEXITY REMINDER

Average O(log n) assumes the BST is reasonably balanced. A skewed BST can take O(n) time for searching, insertion and deletion.

7. AVL Tree

DEFINITION

An AVL Tree is a self-balancing Binary Search Tree (BST) in which the height difference between the left and right subtrees of every node is at most 1.

7.1 AVL Tree Rules

1. Must follow BST properties.
2. Every node must remain height-balanced.
3. Balance Factor must be −1, 0 or +1.

7.2 Height Properties

- Leaf node height = 0
- NULL node height = −1

HEIGHT FORMULA

H(node) = 1 + max(H(L), H(R))

Calculate height from bottom to top.

7.3 Balance Factor (BF)

BALANCE FACTOR FORMULA

BF = H(L) − H(R)

BF	Meaning
+1	Left subtree is taller.
0	Both subtrees have equal height.
−1	Right subtree is taller.
+2	Unbalanced (left-heavy).
−2	Unbalanced (right-heavy).

7.4 Important Exam Formulas

Concept	Value
Leaf height	0
NULL height	−1
Node height	1 + max(L, R)
Balance Factor	Left Height − Right Height
Valid BF	−1, 0, +1
Unbalanced BF	Less than −1 or greater than +1

REMEMBER

Height is calculated bottom to top. Balance Factor = Left − Right. Every node must satisfy the AVL balance condition.

8. AVL Tree Rotations

DEFINITION

AVL rotation is the process of balancing an unbalanced AVL Tree while maintaining the BST property. A node with BF greater than +1 or less than −1 requires rebalancing.

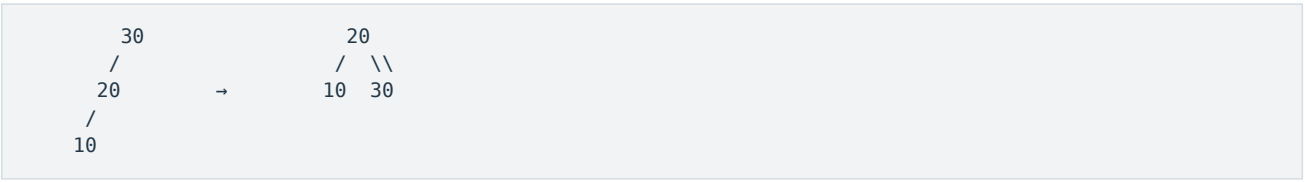
Case	Where imbalance occurs	Rotation required
LL	Left subtree of left child	Right rotation
RR	Right subtree of right child	Left rotation
LR	Right subtree of left child	Left, then right

Case	Where imbalance occurs	Rotation required
RL	Left subtree of right child	Right, then left

8.1 LL Rotation (Left-Left)

- Insertion occurs in the left subtree of the left child.
- Requires a right rotation.

BEFORE → AFTER



8.2 RR Rotation (Right-Right)

- Insertion occurs in the right subtree of the right child.
- Requires a left rotation.

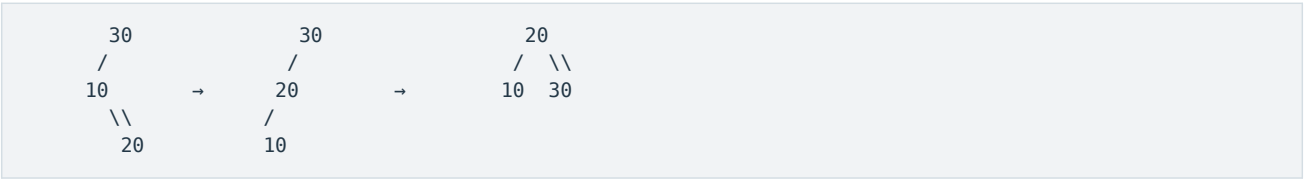
BEFORE → AFTER



8.3 LR Rotation (Left-Right)

- Insertion occurs in the right subtree of the left child.
- Requires two rotations: (1) Left rotation on the left child; (2) Right rotation on the unbalanced node.

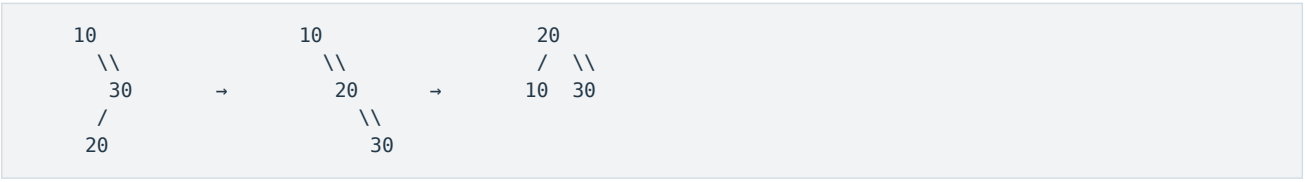
BEFORE → AFTER FIRST ROTATION → FINAL



8.4 RL Rotation (Right-Left)

- Insertion occurs in the left subtree of the right child.
- Requires two rotations: (1) Right rotation on the right child; (2) Left rotation on the unbalanced node.

BEFORE → AFTER FIRST ROTATION → FINAL



8.5 Important Exam Table

Rotation	Insertion Position	Required Rotation
LL	Left of left child	Right
RR	Right of right child	Left
LR	Right of left child	Left → Right
RL	Left of right child	Right → Left

8.6 Important Points

1. LL and RR require single rotations.
2. LR and RL require double rotations.
3. In a double rotation, rotate the child first, then the unbalanced node.
4. Every rotation must maintain the BST property.
5. After rotation, the tree must satisfy the AVL balance condition.

EASY MEMORY TRICK

LL → Right | RR → Left
LR → Left, then Right | RL → Right, then Left

9. AVL Insertion Algorithm

Definition: Inserting a new node using BST rules and balancing the tree if required.

9.1 Steps

1. Insert the new node using normal BST insertion.
2. Update the height of each ancestor node while returning toward the root.
3. Calculate the Balance Factor (BF).
4. Check whether a node is unbalanced (BF > +1 or BF < −1).
5. Identify the rotation case (LL, RR, LR or RL).
6. Perform the required rotation.
7. Return the updated root.

9.2 Pseudocode

```
AVL_INSERT(root, value)

1. If root == NULL:
       Create and return new node.

2. If value < root->data:
       Insert into left subtree.

3. Else if value > root->data:
       Insert into right subtree.

4. Update height of root.

5. Calculate BF of root.

6. If unbalanced:
       Identify LL, RR, LR, or RL.
       Perform required rotation.

7. Return updated root.
```

DUPLICATES IN AVL TREES

The original AVL pseudocode does not specify what to do when value == root->data. A common implementation either rejects duplicate keys or stores a count in the node. Choose one policy consistently; do not assume the BST 'equal goes right' rule automatically remains valid after rotations.

10. AVL Deletion Algorithm

Definition: Deleting a node using BST deletion rules and rebalancing the tree wherever required.

10.1 Three BST Deletion Cases

1. Leaf node → Simply delete it.
2. One child → Replace the node with its child.
3. Two children → Replace with inorder successor or predecessor, then delete the replacement node from its original position.

10.2 Steps

1. Search for the node using BST rules.
2. Delete the node using the three BST deletion cases.
3. Update the height of each ancestor node.
4. Calculate the Balance Factor (BF).
5. Check whether any node is unbalanced.
6. Identify the rotation case (LL, RR, LR or RL).
7. Perform the required rotation.
8. Continue checking ancestors up to the root.
9. Return the updated root.

10.3 Pseudocode

```
AVL_DELETE(root, value)

1. If root == NULL:
       Return NULL.

2. Search for the node using BST rules.

3. Delete the node using BST deletion cases.

4. If root == NULL:
       Return NULL.

5. Update height of root.

6. Calculate BF of root.

7. If unbalanced:
       Identify LL, RR, LR, or RL.
       Perform required rotation.

8. Return updated root.
```

DELETION REMINDER

Unlike insertion, deletion can make multiple ancestors unbalanced. Continue updating heights and rebalancing while moving toward the root.

11. AVL Insertion vs Deletion

Insertion	Deletion
Uses BST insertion.	Uses BST deletion.
Updates heights.	Updates heights.
Calculates BF.	Calculates BF.
Rebalances the first unbalanced ancestor.	May require rebalancing multiple ancestors.
Usually one rebalancing operation is sufficient.	May require multiple rotations.

12. AVL Time Complexity

Operation	Time Complexity
Search	O(log n)
Insertion	O(log n)
Deletion	O(log n)
Single rotation	O(1)

FINAL EXAM SUMMARY

AVL Tree keeps the tree height-balanced. Valid balance factors are −1, 0 and +1. LL uses right rotation; RR uses left rotation; LR uses left then right; RL uses right then left. Search, insertion and deletion take O(log n) time in an AVL tree.